UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE ESTUDIOS SUPERORES ARAGÓN

Manual Técnico

EQUIPO “JUANES”

HABILIDADES DIRECTIVAS

PROF: AARON VELASCO AGUSTIN

GRUPO 2808

SEMESTRE 2026-II

21 DE ABRIL DE 2026

1. Introducción

1.1 Resumen del proyecto

Delivery Plus es una plataforma integral de gestión de pedidos de comida inspirada en modelos de negocio de economía colaborativa como Uber Eats. El sistema ha sido diseñado como una solución académica robusta que simula el ciclo de vida completo de una orden: desde la exploración de menús y la gestión local de productos, hasta la persistencia de pedidos y administración de perfiles de usuario.

1.2 Propósito del Manual

Este documento tiene como objetivo servir de guía técnica para desarrolladores, administradores de sistemas y personal de mantenimiento. Proporciona una visión detallada de la arquitectura de microservicios, la estructura del frontend basada en componentes y el esquema relacional de la base de datos, facilitando la instalación, el despliegue y la futura escalabilidad del software.

1.3 Objetivos del Sistema

- Gestión Multicanal de Datos: Integrar un catálogo dinámico de restaurantes y platillos consumidos desde un backend en tiempo real.
- Experiencia de Usuario (UX) Fluida: Implementar una interfaz moderna y responsiva utilizando Tailwind CSS que garantice la facilidad de uso en dispositivos de escritorio y móviles.
- Seguridad y Persistencia: Asegurar la integridad de las sesiones mediante JSON Web Tokens (JWT) y optimizar el rendimiento del carrito de compras mediante persistencia en el lado del cliente (localStorage).
- Independencia de Módulos: Mantener una separación clara entre la lógica de autenticación y la lógica de negocio mediante una arquitectura de servicios divididos.

1.4 Alcance Técnico

El proyecto abarca tres áreas fundamentales:

1. Frontend (React + Vite): Una Single Page Application (SPA) que gestiona el estado global de la sesión y el carrito.
2. Backend (Node.js): Una infraestructura dividida en auth-service para la seguridad y app-service para la lógica transaccional.
3. Capa de Datos (MySQL): Un modelo relacional que soporta la trazabilidad completa de los usuarios, sus direcciones y su historial de consumo.

2. Arquitectura del sistema

Esta sección detalla la organización lógica y física del proyecto, la cual se basa en una separación clara de responsabilidades para facilitar el trabajo en equipo y el mantenimiento independiente de cada módulo.

2.1 Modelo de Arquitectura

El sistema sigue un modelo de Arquitectura Cliente-Servidor con un enfoque de Microservicios desacoplados en el backend:

- Capa de Presentación (Frontend): Una Single Page Application (SPA) construida en React que gestiona la experiencia del usuario y la lógica del carrito de compras de forma local.
- Capa de Negocio (Backend): Dividida en dos servicios independientes ejecutados en Node.js para separar la seguridad de la lógica operativa.
- Capa de Datos (Persistencia): Un motor relacional MySQL que centraliza la información de usuarios, productos y pedidos.

2.2 Tecnologías del Stack (Tech Stack)

El proyecto utiliza herramientas modernas seleccionadas por su eficiencia y escalabilidad en entornos de desarrollo colaborativo:

Componente	**Tecnología**	**Propósito**
Interfaz de Usuario	React + Vite	Renderizado rápido y gestión de componentes.
Estilos	Tailwind CSS	Diseño responsivo basado en utilidades.
Entorno de Servidor	Node.js	Ejecución de lógica de backend escalable.
Framework Web	Express	Gestión de rutas y Middlewares para la API.
Base de Datos	MySQL	Almacenamiento relacional y consistente de datos.
Comunicación	JWT	Autenticación segura mediante tokens entre capas.

2.3 Flujo de Datos y Comunicación

El intercambio de información entre las capas se realiza mediante peticiones HTTP siguiendo los principios REST:

1. Peticiones del Cliente: El frontend solicita datos a través de servicios específicos (ej. restaurantService.js).
2. Procesamiento en Backend: * El auth-service valida credenciales y entrega un token JWT que el cliente almacena en localStorage.
    - El app-service gestiona las consultas a la base de datos para obtener restaurantes o registrar pedidos.
3. Persistencia Local: A diferencia de una app tradicional, el carrito de compras no requiere peticiones constantes al servidor, ya que la lógica de selección y suma reside enteramente en el navegador del usuario.
4. Respuesta del Servidor: El backend entrega respuestas en formato JSON que el frontend procesa para actualizar la interfaz dinámicamente.

3. Especificaciones del Backend (Node.js)

El backend de Delivery Plus está construido sobre Node.js utilizando el framework Express. Su diseño se basa en la segmentación de responsabilidades mediante dos microservicios principales que se comunican con una base de datos centralizada en MySQL.

3.1 Estructura de Microservicios

La lógica del servidor se divide de la siguiente manera para permitir un desarrollo independiente por parte del equipo:

- auth-service: Se encarga exclusivamente de la gestión de identidad. Maneja el registro de nuevos usuarios, la validación de credenciales y la generación de tokens JWT para mantener sesiones seguras.
- app-service: Centraliza la lógica de negocio de la aplicación, incluyendo la consulta de restaurantes, la gestión de menús, la recepción de pedidos y la administración de perfiles y direcciones.

3.2 Organización del Código

Cada servicio dentro de la carpeta backend/ sigue un patrón organizado para garantizar la escalabilidad:

- Controllers: Contienen la lógica que procesa las solicitudes y define qué datos enviar de vuelta al cliente.

- Models: Definen las consultas SQL y la interacción directa con las tablas de la base de datos.
- Routes: Establecen los endpoints específicos que el frontend consume (ej. /api/auth/login o /api/restaurants).
- Middlewares: Actúan como filtros de seguridad, verificando que el token JWT sea válido antes de permitir el acceso a rutas protegidas como la creación de pedidos o la edición de perfil.

3.3 Definición de Endpoints Principales

A continuación, se detallan las rutas de acceso que el backend expone para ser consumidas por el frontend:

Servicio de Autenticación (auth-service)

Método	Ruta	Descripción
POST	/api/auth/register	Crea un nuevo registro en la tabla users.
POST	/api/auth/login	Valida credenciales y retorna un token JWT.

Servicio de Aplicación (app-service)

Módulo	Ruta	Acción
Restaurantes	GET /api/restaurants	Lista todos los restaurantes disponibles.
Menús	GET /api/restaurants/:id/menu	Obtiene los platillos asociados a un restaurante específico.
Pedidos	POST /api/orders	Registra una nueva orden en las tablas orders y order_items.
Direcciones	POST /api/profile/addresses	Agrega una nueva dirección vinculada al ID del usuario.

3.4 Gestión de la Base de Datos

El sistema utiliza una base de datos relacional con 6 tablas principales: users, restaurants, menu_items, orders, order_items y addresses. El archivo schema.sql define la estructura de estas relaciones, mientras que seed.sql proporciona datos iniciales (como 15 restaurantes y 75 productos) para facilitar las pruebas de desarrollo.

4. Especificaciones del Frontend (React + Vite)

El frontend de Delivery Plus es una Single Page Application (SPA) diseñada para ofrecer una experiencia de usuario rápida y fluida. La interfaz es responsiva y se adapta a diferentes tamaños de pantalla gracias al uso de Tailwind CSS.

4.1 Estructura de Directorios

El código fuente dentro de frontend/src/ sigue una organización modular para facilitar el mantenimiento y la colaboración de los 5 integrantes asignados a esta capa:

- pages/: Contiene los componentes de pantalla completa que representan cada ruta de la aplicación (ej. LoginPage, HomePage, CartPage).
- components/: Alberga piezas de interfaz reutilizables como la barra de navegación (Navbar), tarjetas de restaurantes (Cards) y botones específicos.
- services/: Módulo encargado de realizar las peticiones HTTP a los microservicios del backend utilizando la API Fetch o Axios.
- utils/ o hooks/: Funciones auxiliares para la lógica del carrito y la persistencia de datos.

4.2 Gestión del Estado y Persistencia Local

Una característica técnica clave de este proyecto es cómo maneja los datos sin depender constantemente del servidor:

- Autenticación: Al iniciar sesión, el token JWT se almacena en localStorage. Un componente de orden superior (ProtectedRoute.jsx) verifica la existencia de este token antes de permitir el acceso a rutas privadas.
- Carrito de Compras: Para optimizar el rendimiento y permitir el funcionamiento offline parcial, el carrito vive enteramente en el navegador (localStorage). La lógica de agregar, quitar o sumar productos se gestiona a través de cartService.js.
- Sincronización: Solo cuando el usuario decide finalizar la compra en CheckoutPage, el estado del carrito se envía al backend para crear una orden persistente en la base de datos.

4.3 Componentes Críticos del Sistema

Cada módulo del frontend tiene una responsabilidad clara asignada a los desarrolladores:

Módulo	Componentes Clave	Lógica Principal
Catálogo	HomePage, Navbar	Listado de restaurantes y búsqueda.

Módulo	Componentes Clave	Lógica Principal
Menú	RestaurantPage	Visualización de platillos y acción de añadir al carrito.
Checkout	CartPage, CheckoutPage	Resumen de compra, cálculo de totales y confirmación.
Perfil	ProfilePage, AddressesPage	Edición de datos personales y gestión de direcciones (CRUD).

4.4 Navegación y Rutas

La aplicación utiliza un sistema de enrutamiento dinámico donde:

1. Rutas Públicas: Acceso libre a Login y Registro.
2. Rutas Protegidas: Requieren un token válido para acceder al historial de pedidos (OrdersPage) o a la configuración del perfil.
3. Redirecciones: Si un usuario intenta realizar un pedido sin estar autenticado, el sistema lo redirige automáticamente al flujo de login.

5. Seguridad y Autenticación

La seguridad de la plataforma se basa en un modelo de Autenticación Estateless (sin estado) utilizando el estándar JSON Web Tokens (JWT). Esto permite que el servidor no tenga que recordar cada sesión activa, delegando la responsabilidad de la identidad al token que porta el cliente.

5.1 Flujo de Autenticación JWT

El proceso de acceso sigue un flujo estrictamente definido para proteger las credenciales del usuario:

1. Solicitud de Acceso: El usuario envía su email y contraseña a través de LoginPage.jsx al endpoint POST /api/auth/login.
2. Validación y Generación: El auth-service verifica las credenciales contra la base de datos MySQL. Si son correctas, genera un token JWT firmado con una clave secreta del lado del servidor.
3. Almacenamiento en Cliente: El frontend recibe el token y lo almacena en el localStorage del navegador para persistir la sesión incluso si se refresca la página.
4. Autorización de Peticiones: En cada solicitud posterior a rutas protegidas (como crear un pedido o ver el perfil), el frontend adjunta el token en los encabezados (Headers) de la petición HTTP.

5.2 Protección de Rutas en el Frontend

Para evitar que usuarios no autenticados accedan a secciones privadas, se implementa un componente especializado:

- ProtectedRoute.jsx: Este componente actúa como una "guarda" o filtro. Antes de renderizar pantallas como OrdersPage o ProfilePage, verifica la presencia y validez aparente del token en el localStorage.
- Redirección Automática: Si el token no existe, el sistema redirige automáticamente al usuario a la pantalla de Login, protegiendo la privacidad de los datos.

5.3 Seguridad en el Backend (Middlewares)

El servidor no confía ciegamente en las peticiones del frontend. Utiliza Middlewares de Autenticación:

- Antes de procesar peticiones en el app-service (como PUT /api/profile), un middleware intercepta la llamada y decodifica el JWT.
- Si el token ha expirado o la firma es inválida, el servidor responde con un código de error 401 (Unauthorized), bloqueando cualquier operación en la base de datos.

5.4 Integridad del Carrito

Aunque el carrito de compras reside en el localStorage para mayor velocidad, la seguridad se aplica al momento del Checkout:

- El sistema valida que el usuario que intenta realizar el pedido sea el mismo que posee el token activo.
- Los precios y productos se verifican contra la base de datos en el servidor al momento de la compra para evitar manipulaciones de precio desde el navegador.

6. Configuración e Instalación

Esta sección describe el procedimiento para replicar el entorno de desarrollo localmente. Es fundamental seguir el orden establecido para asegurar que la comunicación entre los microservicios y el frontend sea exitosa.

6.1 Requisitos Previos

Antes de comenzar, asegúrate de tener instaladas las siguientes herramientas:

- Node.js: Versión 18 o superior.
- Gestor de paquetes: npm (incluido con Node.js).

- Base de Datos: MySQL Server corriendo localmente.
- Entorno de desarrollo: Visual Studio Code (recomendado).

6.2 Preparación de la Base de Datos

El proyecto incluye scripts SQL para automatizar la creación de la estructura y la carga de datos iniciales:

1. Crear el Schema: Ejecuta el archivo database/schema.sql en tu gestor de MySQL para crear las tablas users, restaurants, menu_items, orders, order_items y addresses.
2. Cargar Semillas (Seeds): Ejecuta database/seed.sql para precargar 15 restaurantes y 75 productos de prueba.

6.3 Configuración del Backend (Microservicios)

Debes configurar e iniciar ambos servicios (auth-service y app-service) de forma independiente:

1. Navegar a la carpeta: Accede a cada subcarpeta dentro de backend/.
2. Instalar dependencias: Ejecuta el comando npm install en cada servicio.
3. Variables de Entorno: Crea un archivo .env en cada carpeta de servicio con las credenciales de tu base de datos local y la clave secreta para el JWT.
4. Iniciar Servidores: Ejecuta npm start o npm run dev.

6.4 Configuración del Frontend

1. Instalar dependencias: Desde la carpeta frontend/, ejecuta npm install.
2. Configurar Endpoints: Abre la carpeta src/services/ y asegúrate de que las URLs de conexión apunten a los puertos correctos de tus microservicios locales (usualmente localhost:3000 o similar).
3. Lanzar la App: Ejecuta npm run dev para iniciar el servidor de desarrollo de Vite.

6.5 Verificación de la Instalación

Una vez completados los pasos anteriores, deberías ser capaz de:

- Acceder a la aplicación desde el navegador (típicamente en http://localhost:5173).
- Ver la lista de restaurantes cargada desde la base de datos.
- Registrar un nuevo usuario y observar la creación del registro en la tabla users de MySQL.

7. Mantenimiento y Escabilidad

Esta sección proporciona las directrices para que futuros desarrolladores puedan depurar, optimizar y expandir las capacidades de Delivery Plus de manera ordenada.

7.1 Pruebas y Validación (Testing)

Dada la naturaleza modular del proyecto, se recomiendan los siguientes niveles de prueba:

- Pruebas de API: Utilizar herramientas como Postman o Insomnia para verificar que los endpoints de auth-service y app-service retornen los códigos de estado correctos (200 OK, 201 Created, 401 Unauthorized).
- Validación de Persistencia: Verificar regularmente que la lógica del cartService.js mantenga la integridad de los productos en el localStorage ante cierres inesperados del navegador.
- Consistencia de Datos: Asegurar que, al eliminar una dirección en el frontend, el cambio se refleje correctamente en la tabla addresses de MySQL mediante el endpoint DELETE correspondiente.

7.2 Monitoreo de Errores Comunes

El manual debe documentar soluciones a fallos frecuentes identificados durante el desarrollo:

- Errores de CORS: Problemas de comunicación entre el frontend (Vite) y el backend (Node.js) si los puertos no están correctamente autorizados en los middlewares de Express.
- Token Expirado: Manejo de la lógica para redirigir al usuario al login cuando el JWT almacenado deja de ser válido.
- Sincronización de Base de Datos: Asegurarse de que el servidor MySQL esté activo antes de iniciar los servicios de Node.js para evitar errores de conexión (ECONNREFUSED).

7.3 Hoja de Ruta para la Escalabilidad

Para transformar este proyecto académico en una solución lista para producción, se sugieren las siguientes mejoras:

- **Infraestructura:**
    - Migrar de localStorage a un estado global con Redux o Context API para una gestión del carrito más robusta.
    - Implementar Docker para contenerizar los microservicios, facilitando su despliegue en la nube (AWS/Azure).

- **Funcionalidades:**
    - Pasarela de Pagos: Integrar APIs reales como Stripe o PayPal en el CheckoutPage.
    - Notificaciones: Implementar WebSockets o Firebase Cloud Messaging para avisar al usuario cuando su pedido cambie de estado en tiempo real.
    - Optimización de Imágenes: Implementar un servicio de almacenamiento en la nube (como Cloudinary) para las fotos de los restaurantes en lugar de cargarlas localmente.

7.4 Documentación de Cambios

Se recomienda mantener un archivo CHANGELOG.md donde cada uno de los 10 integrantes del equipo registre las modificaciones realizadas en sus respectivos módulos (Auth, Catálogo, Pedidos, etc.), evitando conflictos de código y facilitando el rastreo de versiones.